



SOFTWARE TESTING · BACKEND QUALITY — DETAILED EDITION

Database Testing

Fundamentals · data generation · unit testing · migration — methodology, SQL & the AI state of the art

Consolidated from FIT lecture decks + verified tool/technique survey (2024–2026)
Detailed edition: runnable SQL and framework code on every technique

Four parts, each with runnable code



1 • Fundamentals

Structural, functional, ACID, recovery & security — with SQL you can run: constraint tests, EXPLAIN, integrity queries, trigger tests, isolation experiments.



2 • Data Generation

Why realistic data matters, the difficulties in depth, five generation methods with tradeoffs, Faker & generate_series, masking, and AI synthetic data.



3 • Unit Testing

The known-state loop and real framework code: tSQLt, pgTAP, dbt tests, Great Expectations, Soda Core.



4 • Migration & SOTA

Expand/contract migrations, LLM-to-SQL reality check, ML data observability, and SQLancer metamorphic oracles.

PART 1

Fundamentals

The database is where the truth lives — the UI can look perfect while the data underneath is silently wrong.

What & why — and the five types

Verify the correctness and integrity of the data behind the app. DB defects don't throw exceptions — only queries and tests reveal them.



Functional

Tables, queries, stored procedures & transactions produce correct results.



Structural

Schema itself: tables, keys, constraints, indexes, referential integrity.



Non-functional

Performance, security, reliability, responsiveness under load.



Data integrity

No loss on change; transactions all-or-nothing; PK/FK preserved.



Recovery

Backup runs; restore works; integrity intact; RTO met.

PRINCIPLE Test the database as its own artifact — not only through the UI. Bugs hide where the UI never looks.

Functional — is the answer correct?

Tables, columns, queries, stored procedures and transactions all produce the results the business rules require.

WHAT IT CHECKS

- › Tables & columns behave correctly — types, defaults, computed values
- › Query results are consistent and complete — every required row returned
- › Transactions reach the correct end-state (commit / rollback as specified)
- › Stored-procedure logic matches the business rules it encodes

EXAMPLE — ASSERT THE RESULT, NOT THE UI

```
-- Rule: count active premium users
SELECT count(*) AS n FROM users
WHERE status = 'active'
      AND plan  = 'premium';
-- expected: 1240  (matches the fixture)

-- Proc must apply a 10% loyalty discount
CALL apply_discount(order_id => 501);
SELECT total FROM orders WHERE order_id=501;
-- expected: 90.00  (was 100.00)
```

WHY IT HIDES A wrong WHERE clause still returns rows — just the wrong ones. The screen looks fine; only a query with a known expected result catches it.

Structural — is the schema itself correct?

The schema — tables, columns, keys, constraints and indexes — is defined and enforced exactly as designed.

WHAT IT CHECKS

- › Table & column definitions match the data model
- › Indexes exist on the columns queries actually filter and join
- › Primary keys and foreign keys are declared and correct
- › CHECK, NOT NULL and UNIQUE constraints reject bad data
- › Referential integrity holds across every relationship

EXAMPLE — SCHEMA REJECTS & INDEX EXISTS

```
-- FK must reject an unknown parent
INSERT INTO orders(order_id, customer_id)
VALUES (1, 999);
-- ERROR: foreign key violation
--      SQLSTATE 23503

-- The index the report needs must exist
SELECT indexname FROM pg_indexes
WHERE tablename = 'orders'
  AND indexdef LIKE '%customer_id%';
-- expected: 1 row
```

REJECTION IS THE PASS A structural test passes when the DBMS raises the expected error — the deep-dive slides that follow assert on the exact SQLSTATE.

Non-functional — fast, secure, reliable

The qualities the schema can't declare but the system must meet: performance, security, reliability and responsiveness under load.

WHAT IT CHECKS

- › Performance — query latency and throughput under realistic load
- › Security — access control, encryption, SQL-injection resistance
- › Reliability — correct behaviour under failure and heavy concurrency
- › Responsiveness — stays within its latency target during peaks

EXAMPLE — MEASURE VS A TARGET, PROBE INJECTION

```
-- Performance: latency budget for a query
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id=42;
-- Index Scan ... actual time 0.02 ms
-- assert: p95 < 200 ms @ 500 users

-- Security: payload must be treated as data
-- input: ' OR '1'='1
-- bound parameters -> 0 rows, no leak
```

MEASURE, DON'T GUESS 'Feels fast' is not a result. Assert a number — $p95 < 200$ ms at 500 concurrent users — so any regression is unambiguous.

Data integrity — no loss, no half-truth

Data survives every change intact: nothing lost, transactions all-or-nothing, primary and foreign keys preserved.

WHAT IT CHECKS

- › No data loss when rows are inserted, updated or deleted
- › Every transaction completes fully or not at all — never half-applied
- › Store and retrieve stay consistent — what went in comes back
- › Values remain accurate after each change
- › Primary and foreign keys are maintained across operations

EXAMPLE — QUERIES WRITTEN TO RETURN 0 ROWS

```
-- Orphaned children (FK gap after import)
SELECT o.order_id FROM orders o
LEFT JOIN customers c
  ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;  -- expect 0

-- Duplicates on a business key
SELECT email, count(*) FROM users
GROUP BY email HAVING count(*)>1; -- 0
```

ZERO IS GREEN Write each integrity check to return zero rows on success — it wires straight into an assertion with no extra logic.

Recovery — can you get everything back?

Backups run and are restorable, integrity survives the restore, and recovery time meets the agreed RTO.

WHAT IT CHECKS

- › The backup procedure runs, is automated, and produces restorable artifacts
- › Data is fully restorable after a failure or corruption
- › Integrity is intact after restore — row counts and checksums match
- › Recovery time meets the RTO; data loss stays within the RPO

EXAMPLE — A RESTORE DRILL YOU CAN ASSERT ON

```
# 1. Back up, then simulate disaster
pg_dump prod > backup.sql
# ... drop / corrupt the database ...

# 2. Restore into a clean instance
psql restore_test < backup.sql

-- 3. Verify nothing was lost
SELECT count(*) FROM orders;
-- must equal pre-failure count; time < RTO
```

AN UNTESTED BACKUP ISN'T ONE Backups fail silently. Only a scheduled restore drill proves the data — and the clock — actually meet the RTO.

Structural tests — the schema rejects bad data

A structural test passes when the DBMS raises the expected error.

REJECTION IS THE PASS CONDITION

```
-- FK violation test (expected: rejection)
INSERT INTO orders(order_id, customer_id)
VALUES (1, 999);
-- ERROR: violates foreign key constraint
-- "orders_customer_id_fkey"
-- SQLSTATE 23503

-- CHECK violation test
INSERT INTO accounts(id, balance) VALUES (1,-50);
-- ERROR: violates check constraint
-- SQLSTATE 23514
```

SQLSTATE codes to assert on:

Code	Meaning
23503	foreign_key
23514	check
23505	unique
23502	not_null

ASSERT BOTH In a harness, assert the statement failed AND that the SQLSTATE matches — a different error is still a bug.

Is my index actually used?

EXPLAIN shows the plan; the test asserts Index Scan, not Seq Scan.

READING THE QUERY PLAN

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 42;

-- Index Scan using orders_customer_id_idx on orders
--   (cost=0.42..8.44 rows=1 width=36)
--   (actual time=0.021..0.023 rows=1 loops=1)
```

SEQ SCAN = RED FLAG A Seq Scan on a large table means the index is missing, unusable, or your predicate is non-sargable — e.g. WHERE lower(email)=... with no matching functional index.

Rule: index the columns your WHERE / JOIN / ORDER BY actually use — and prove it with EXPLAIN, don't assume.

Data-integrity queries — written to return 0 rows

Each query returns nothing on success — trivial to wire into an assertion.

THE THREE INTEGRITY WORKHORSES

```
-- Orphaned child rows (FK gap after a bulk import)
SELECT o.order_id
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;      -- parent missing => orphan

-- Duplicates on a business key
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- Mandatory column must never be NULL
SELECT COUNT(*) FROM users WHERE email IS NULL;    -- expect 0
```

Testing triggers & stored procedures

Test both the firing path and the non-firing path.

A BUSINESS-RULE TRIGGER

```
CREATE FUNCTION prevent_negative_balance()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.balance < 0 THEN
        RAISE EXCEPTION 'balance cannot be negative'
        USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_balance BEFORE UPDATE
ON accounts FOR EACH ROW
EXECUTE FUNCTION prevent_negative_balance();
```

THE TEST

```
-- TEST: must be rejected
UPDATE accounts
    SET balance = balance - 1000
    WHERE id = 1;
-- ERROR: balance cannot
-- be negative
-- SQLSTATE 23514
```



Also test

The allowed path succeeds; the trigger doesn't fire twice; no infinite trigger loops; performance stays acceptable.

ACID 1 — atomicity, tested concretely

ATOMICITY — ALL OR NOTHING

```
BEGIN;  
  UPDATE accounts SET balance = balance - 100  
    WHERE id = 1;  
  UPDATE accounts SET balance = balance + 100  
    WHERE id = 2;  
ROLLBACK;  
  
-- ASSERT: balances of id 1 and 2 equal  
-- their pre-transaction values (nothing stuck)
```



The oracle

Snapshot balances before; run the transaction; force an error or ROLLBACK; re-read. Either both rows changed or neither did.



Where it breaks

A procedure that catches an error mid-way and continues — leaving a half-applied transfer. That's the bug atomicity tests catch.

TIP

Wrap the whole test in a transaction you ROLLBACK at the end — the database returns to a pristine, deterministic state for the next test.

ACID 2 — isolation & the lost update

Concurrency bugs only appear when two sessions race on the same row.

PESSIMISTIC LOCK FIXES LOST UPDATE

```
-- Sessions A and B run concurrently:
BEGIN;
SELECT balance FROM accounts
  WHERE id = 1 FOR UPDATE;    -- B blocks
                               -- until A commits
UPDATE accounts SET balance = balance + 50
  WHERE id = 1;
COMMIT;
```

Without FOR UPDATE: both read 100, both write 150 — one +50 is lost.

ISOLATION LEVELS vs ANOMALIES

Level	Dirty	Non-rep	Phantom
READ UNCOMM.	Y	Y	Y
READ COMM.	—	Y	Y
REPEAT. READ	—	—	Y
SERIALIZABLE	—	—	—

Y = anomaly possible. ANSI maximums — real engines (e.g. PostgreSQL) are often stricter.

TEST DESIGN Pick two transactions that must not interleave, run them concurrently at each isolation level, and assert the outcome matches the level's contract.

Recovery, backup & security

Non-functional — and the first things audited after an incident.



Backup works

Procedure runs, is automated, and produces restorable artifacts.



Restore verified

Restorable after failure; integrity intact; recovery time meets RTO.



Access control

Roles enforced, least privilege; no unauthorized read/write.



Encryption & injection

Encrypted at rest & in transit; parameterized inputs — SQL injection fails.

INJECTION TEST

Send ' OR '1'='1 and '; DROP TABLE users;-- into every input; a correct app uses bound parameters and treats them as literal data.

PART 2

Test Data Generation

Realistic data at volume — the most important, most time-consuming ingredient of database testing.

Why you need realistic test data



Load & performance

A query fast on 1,000 rows can collapse on 50 million. Performance testing is impossible without volume.



Edge cases

Empty strings, Unicode, boundary dates, NULLs, maximum-length fields — the values that trigger real bugs.



Demo & staging

Realistic-looking data keeps testers engaged and makes features demoable.



Privacy / GDPR

Using a copy of production in dev/test is a regulated data-processing activity. Anonymized or synthetic data removes that legal exposure.

The difficulties — in depth

Why “just use random()” produces data that hides your bugs.



Volume & variety

Enough rows, and enough different rows, to exercise every path.



Referential integrity

A generated `orders.customer_id` must point to a real generated customer — across every FK.



Realistic distributions

Real data is skewed (Zipfian, power-law).
Uniform-random data hides the performance cliffs skew causes.



Uniqueness & constraints

Emails, national IDs, order numbers must satisfy UNIQUE and CHECK while still looking real.



Temporal consistency

$\text{ship_date} \geq \text{order_date} \geq \text{signup_date}$.
Independent random dates break causality.



PII, locale & schema drift

Vietnamese names & diacritics (Nguyễn, Trần),
+84 phone formats; regenerate when the schema changes.

Five generation methods — tradeoffs

Method	Strength	Weakness
Manual	Precise, great for targeted edge cases	Does not scale at all
Rule/template (Faker)	Fast, controllable, locale-aware	Weak inter-column correlation
SQL scripts (generate_series)	Zero deps, extremely fast bulk load	Limited realism
Prod subsetting + masking	Real distributions & correlations	Anonymization effort; re-id risk
Model-based / GAN / LLM	Learns joint distribution automatically	Needs data & compute; leakage risk

RULE OF THUMB

Faker for greenfield unit tests · subsetting+masking for realistic staging · synthetic AI when privacy blocks using

production.

Code: Faker + generate_series

Two workhorses you can use today — one for variety, one for raw volume.

FAKER (PYTHON) — LOCALE-AWARE VARIETY

```
from faker import Faker
fake = Faker("vi_VN")    # Vietnamese locale
rows = [(
    fake.name(),
    fake.unique.email(),
    fake.phone_number(),
    fake.date_of_birth(minimum_age=18,
                       maximum_age=70))
    for _ in range(10_000)]
```

generate_series — RAW BULK VOLUME

```
-- PostgreSQL: 1M rows in one statement
INSERT INTO users (id, email, created_at)
SELECT g,
       'user' || g || '@example.com',
       NOW() - (random() * INTERVAL '365 days')
FROM generate_series(1, 1000000) AS g;
```

COMBINE THEM Faker gives believable names/emails/dates; generate_series gives millions of rows instantly. Use Faker to seed a template table, then fan out with SQL.

For correlated, multi-table realism, move up to a synthetic-data model (next slides).

Production subsetting + masking

Maximum realism from real data — made safe before it leaves production.



Substitution

Replace a real value with a realistic fake (name → fake name).



Shuffling

Permute a column across rows — keeps the distribution, breaks the row linkage.



Nulling / redaction

Overwrite with NULL or XXX where the value isn't needed.



Format-preserving encryption

Ciphertext keeps length & charset (16-digit card stays 16 digits); reversible with a key.



k-anonymity

Generalize/suppress quasi-identifiers so each record is indistinguishable from $\geq k-1$ others.



Differential privacy

Add calibrated noise so any one individual changes outputs by at most a bounded factor ϵ .

AI synthetic data — how and who

Generative models learn the joint distribution, then emit safe lookalikes.



How it works

GANs train a generator vs a discriminator; transformer/diffusion tabular models learn the joint distribution, so correlations (age $\leftrightarrow$ income, city $\leftrightarrow$ postcode) are preserved automatically.



Open source (free)

SDV (MIT): GaussianCopula, CTGAN, HMASynthesizer for multi-table. MOSTLY AI SDK: pip install mostlyai, local training with a differential-privacy option.



Commercial

Tonic.ai (Structural/Textual/Fabricate) · Gretel — acquired by NVIDIA (~\$320M, Mar 2025) · MOSTLY AI with fidelity/utility/privacy reports.



The tradeoff

Higher fidelity $\rightarrow$ higher utility but greater risk the model memorized and reproduces a real record (leakage). Measure both.

Code: multi-table synthesis with SDV

Keeps foreign keys intact across tables — the hard part of fake data.

SDV HMASynthesizer — RELATIONAL SYNTHESIS

```
from sdv.multi_table import HMASynthesizer
from sdv.metadata import Metadata

# data: dict of related DataFrames
metadata = Metadata.detect_from_dataframes(data)
synth = HMASynthesizer(metadata)
synth.fit(data)

# 1.5x the original volume, FKs intact
synthetic = synth.sample(scale=1.5)
```

Evaluate the result on two axes:

Utility — does a model trained on synthetic perform like one trained on real?

Privacy — distance-to-closest-record & membership-inference tests.

STILL A GAP Synthetic models rarely reproduce the weird edge cases — bad encodings, orphaned rows — that real production bugs hinge on. Keep a few hand-crafted evil rows.

PART 3

Database Unit Testing

Bring the discipline of unit testing to schemas, procedures and data logic.

The known-state loop + goals

1 • Reset

Script sets a known state before each test

2 • Act

Run the unit: procedure, trigger, query

3 • Assert

Check data, side effects & errors

4 • Clean

Restore state — or run in a transaction and ROLLBACK

GOALS

- Establish a baseline state of the DB
- Verify every subsequent change against it
- Enable true regression testing
- Catch accidentally dropped / renamed objects

CHALLENGES

- Keeping schema & tests in sync
- Maintaining reference and test data
- Keeping tests independent & order-free
- Concurrent devs sharing one DB; build speed

Framework code 1 — tSQLt & pgTAP

Both run each test in a transaction that auto-rolls back.

tSQLt (SQL SERVER) — FakeTable ISOLATES LOGIC

```
EXEC tSQLt.NewTestClass 'AccountTests';
GO
CREATE PROCEDURE AccountTests.
    [test withdraw reduces balance] AS
BEGIN
    EXEC tSQLt.FakeTable 'dbo.Accounts';
    INSERT INTO dbo.Accounts(AccountId,Balance)
        VALUES (1, 100);
    EXEC dbo.Withdraw @AccountId=1, @Amount=40;
    DECLARE @b MONEY =
        (SELECT Balance FROM dbo.Accounts
         WHERE AccountId=1);
    EXEC tSQLt.AssertEquals 60, @b;
END;
```

pgTAP (POSTGRES) — TAP OUTPUT

```
BEGIN;
SELECT plan(3);
SELECT has_table('public','users',
    'users table exists');
SELECT col_not_null('users','email',
    'email is NOT NULL');
SELECT results_eq(
    'SELECT count(*)::int FROM users
     WHERE active',
    'VALUES (2)', 'two active users');
SELECT * FROM finish();
ROLLBACK;
```

Framework code 2 — dbt tests

Declarative tests live next to the model; dbt 1.8 adds true unit tests.

GENERIC TESTS (schema.yml)

```
# schema.yml – generic tests
models:
  - name: orders
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: status
        tests:
          - accepted_values:
              values: ['placed', 'shipped',
                    'completed', 'cancelled']
  - name: customer_id
    tests:
      - relationships:
          to: ref('customers')
          field: customer_id
```

SINGULAR TEST

```
-- singular test: any row = failure
SELECT order_id
FROM {{ ref('orders') }}
WHERE amount < 0
```

UNIT TEST (given / expect)

```
unit_tests:                # dbt 1.8+
  - name: total_includes_tax
    model: stg_orders
    given: [{input: ref('raw'),
              rows: [{sub: 100, rate: 0.1}]}]
    expect: {rows: [{total: 110}]}
```

Framework code 3 — GX & Soda Core

Declarative data-quality suites, runnable in CI.

GREAT EXPECTATIONS (1.0, Aug 2024 API)

```
import great_expectations as gx
ctx = gx.get_context() # GX Core 1.x
batch = (ctx.data_sources.add_pandas("p")
        .add_dataframe_asset("users")
        .add_batch_definition_whole_dataframe("b")
        .get_batch(batch_parameters={"dataframe":df}))

batch.validate(gx.expectations
               .ExpectColumnValuesToNotBeNull(
                 column="email"))
batch.validate(gx.expectations
               .ExpectColumnValuesToBeBetween(
                 column="age", min_value=0, max_value=120))
```

SODA CORE

```
# checks.yml (SodaCL)
checks for customers:
  - row_count > 0
  - missing_count(email) = 0
  - duplicate_count(email) = 0
  - invalid_percent(phone) < 1:
      valid format: phone number
```

KNOWN-STATE tSQLt and pgTAP wrap each test in BEGIN ... ROLLBACK automatically — guaranteeing test independence and order-independence.

Choosing a framework

Layer	Tool	You write
In-database unit tests	tSQLt · utPLSQL · pgTAP	Assertions with fake tables, run in CI
Pipeline / model tests	dbt Core (generic, singular, unit)	YAML + SQL against mocked inputs
Expectation suites	Great Expectations · dbt-expectations	Declarative Expectations on data
Data-quality checks	Soda Core (+ Cloud “Ask AI”)	SodaCL YAML — or an English prompt

ALL FREE Every tool above is free / open source — a full DB testing stack costs a student nothing.

PART 4

Migration & the State of the Art

Schema changes without downtime, LLMs writing SQL, ML watching data, fuzzers hunting the DBMS itself.

Migration testing — expand / contract

Zero-downtime schema change, verified by counts and checksums.



Expand

Add the new structure (nullable column / table); dual-write to old + new; backfill existing rows.



Contract

Once all readers use the new structure, drop the old. Reversible at every step.

ROW-COUNT + CHECKSUM PARITY

```
-- Validate source vs target after migration
SELECT count(*) FROM source.orders;  -- must match
SELECT count(*) FROM target.orders;
SELECT md5(string_agg(id||col, ',' ORDER BY id))
FROM target.orders;                  -- checksum
```



pgroll (Postgres)

Exposes two schema versions via views + sync triggers; instant rollback.



gh-ost / Atlas

Online MySQL changes; Atlas lints destructive changes in CI.

LLMs writing SQL — a reality check

Great assistant, unreliable authority. Numbers to remember.



Spider 1.0 — solved

Classic text-to-SQL: strong agents $\approx 91\%$. On toy schemas, generated test queries mostly just work.



BIRD — realistic

Messy real DBs: top agents $\approx 73\text{--}81\%$ vs 92.96% human. The gap is closing but real.



Spider 2.0 — not solved

Enterprise BigQuery/Snowflake, ~ 800 -column schemas, multi-step SQL: best agents $\approx 21\%$, GPT-4o $\approx 10\%$.



Caveat

2025–26 studies found high annotation-error rates in these benchmarks — leaderboard ranks are noisy.

LESSON

Generated test SQL is code: review and test it like any other. Hallucinated column names are the classic failure.

LLM-assisted checks + ML observability

NATURAL LANGUAGE → SodaCL

```
# Soda "Ask AI": English -> checks  
"email must never be null and unique"  
-> missing_count(email) = 0  
-> duplicate_count(email) = 0
```



Data observability

Monte Carlo, Metaplane (→ Datadog, 2025): ML monitors on freshness, volume, distribution alert before users notice. Free: Elementary CE (dbt-native).



Data diffing

HALLUCINATION RISK An LLM may invent a column that isn't in the schema, so a "passing" generated suite may be silently testing nothing. Validate every generated check against live metadata first.

OBSERVABILITY LIMIT ML monitors detect after data lands (not prevention) and fatigue on seasonal data; migration tools verify mechanics, not semantic equivalence.

Research: testing the DBMS itself (SQLancer)

Your app trusts the engine — researchers don't. Metamorphic oracles, no known answer needed.



PQS — Pivoted Query Synthesis

Pick a random “pivot” row; synthesize a query guaranteed to return it. If the DBMS omits it → bug. (~121 bugs.)



NoREC — Non-optimizing Reference

Run a query the optimizer accelerates, then an equivalent form it can't optimize. Different results → an optimization bug. (~51 bugs.)



TLP — Ternary Logic Partitioning

Split a predicate into p / NOT p / p IS NULL; the union of the three must equal the unpartitioned result. Works for aggregates too. (~77 bugs.)



Why it matters

450+ real logic bugs found in SQLite, MySQL, PostgreSQL... Runs on your laptop in minutes — a great, free research project.

ALSO EMERGING LLM-based DBMS fuzzing (Fuzz4All '24, ShQveL '25) and structure-aware SQL mutation testing — a live thesis area.

A free student stack for DB testing

Need	Tool	Cost
Hunt DBMS logic bugs	SQLancer on SQLite	Free (open source)
In-DB unit tests	pgTAP · tSQLt · utPLSQL	Free
Pipeline / model tests	dbt Core + dbt-expectations	Free
Expectation / DQ suites	Great Expectations · Soda Core	Free
Observability	Elementary Community Edition	Free
Synthetic & fake data	SDV (MIT) · MOSTLY AI SDK · Faker	Free (open source)
Safe migrations	pgroll · gh-ost · Atlas CI	Free

EXERCISE

Your turn: test the data layer

Fundamentals + SQL

Write the 3 integrity queries + an atomicity ROLLBACK test for a MoMo-style wallet transfer. Prove an index is used with EXPLAIN.

Data generation

Generate 10k Vietnamese-locale students with Faker, fan out to 1M enrolments via `generate_series`, then synthesize a related schema with SDV (FKs intact).

Frameworks + SOTA

Write 5 dbt-expectations checks for a Moodle gradebook; then run SQLancer on SQLite for 10 min and explain one bug-triggering query's oracle.

Deliverable: a repo with SQL + tests and a note on what AI generated vs what you had to fix.